



GitHub Trending Top 10

每日热门开源项目深度解读

2026-08-04

今日榜单

- 1 TencentCloud/TencentDB-Agent-Memory** TypeScript 3天
TencentDB Agent Memory is a team-level memory hub for AI Agents — turning conver...
- 2 zhaoxuya520/reverse-skill** PowerShell 5天
Reverse Engineering / Authorized Penetration Testing / Security Research Skill R...
- 3 firecrawl/pdf-inspector** Rust 2天
Fast Rust library for PDF inspection, classification, and text extraction. Intel...
- 4 uber/ADR** Python NEW
ADR secures enterprise AI agents through observability, security benchmarking, a...
- 5 obra/superpowers** Shell NEW
An agentic skills framework & software development methodology that works.
- 6 microsoft/generative-ai-for-beginners** Jupyter Notebook 3天
21 Lessons, Get Started Building with Generative AI
- 7 cypress-io/cypress** TypeScript NEW
Fast, easy and reliable testing for anything that runs in a browser.
- 8 lyogavin/airllm** Jupyter Notebook 3天
AirLLM 70B inference with single 4GB GPU
- 9 webpack/webpack** JavaScript NEW
A bundler for javascript and friends. Packs many modules into a few bundled asse...
- 10 gabime/spdlog** C++ NEW
Fast C++ logging library.

#1

TypeScript

连续 3 天

TencentDB-Agent-Memory

TencentDB Agent Memory is a team-level memory hub for AI Agents — turning conversations, docs, and code into four reusable memory assets (Chat Memory, Skill, LLM-Wiki, Code-Graph) that are governed, shared, and equipped across agents and frameworks.

Stars **12,748** Forks **1,210** Today **+1,090**

<https://github.com/TencentCloud/TencentDB-Agent-Memory>

项目简介：

这是腾讯云推出的“Agent 团队级记忆中枢”，核心解决 AI Agent 反复“失忆”、重复劳动的问题。它把对话、文档、代码自动转化为可复用的记忆资产，让经验在 Agent 团队中沉淀、流转，而不是每次从零开始。

核心功能：

1. **四类记忆资产**：聊天记忆（偏好/决策）、技能库（Skill）、文档知识库（LLM-Wiki）、代码图谱（Code-Graph）。
2. **自动提取与蒸馏**：对话按 L0→L3 层级提炼，从原始对话到人设画像，层层抽象。
3. **框架无关、跨 Agent 共享**：记忆资产与具体 Agent 框架解耦，可在 Claude、CodeBuddy 等不同工具间迁移使用。
4. **冷启动友好**：支持导入已有文档、代码库和历史会话，新团队第一天就能站在前人经验上工作。

适用场景：

- 个人开发者用多个 Agent 管理项目，希望“一次解释，永久记住”。
- 小型团队构建共享 Agent 工作流，例如统一维护代码规范、业务知识库。
- 需要长期积累领域经验的企业，如客服、运维、研发团队，减少重复沟通成本。

技术亮点：

- 采用 **memory-core + memory-hub + proxy** 三服务架构，一键启动，开箱即用。
- 记忆资产以结构化数据存储（非简单 key-value），支持分层蒸馏，兼顾细节与泛化。
- 提供 v2→v3 数据迁移工具，体现工程成熟度；TypeScript 实现，对 Node 生态友好。

上手建议：

- 新手：直接 `git clone` 后按 README 运行 `./start-all.sh`，填好两组 LLM 参数，几分钟内即可

在本地面板体验。

- 开发者：先阅读 `INSTALL.md` 了解部署细节，再从 `MemoryCore/scripts` 下的迁移工具或示例代码入手，理解记忆资产的数据结构。
- 贡献者：关注 GitHub Issues 和 Discord 社区，优先从“新记忆类型支持”或“更多框架适配”方向切入。

#2

PowerShell

连续 5 天

reverse-skill

Reverse Engineering / Authorized Penetration Testing / Security Research Skill Router
Pack AI-powered routing + On-demand toolchain bootstrapping + Self-evolving
knowledge base Supports Claude Code, Kiro, Cursor, Cline, and other AI coding
clients 逆向/渗透/安全技能路由包 - AI 自动路由 + 按需自举工具链 + 自动进化经验库 | 支持
Claude Code / Kiro / Cursor / Cline 等代码 AI 客户端

Stars **17,081** Forks **2,371** Today **+2,446**

<https://github.com/zhaoxuya520/reverse-skill>

项目简介：

reverse-skill 是一个面向 AI 编程助手的“安全技能路由包”，它把逆向工程、授权渗透测试和安全研究的方法论、工具链与经验库打包成结构化规则，让 AI 在遇到 APK、二进制、前端加密或 CTF 任务时，能自动选择正确的工具和流程，而不是瞎猜命令。简单说，它给 AI 装了一套“网络安全操作手册”。

核心功能：

1. **AI 自动路由**：根据任务类型（APK、ELF、JS、PCAP 等）自动匹配对应的技能路径和工具链。
2. **按需工具自举**：自动检测并加载本机已有的工具（如 jadx、Frida、Ghidra），缺失时提示安装或引导配置。
3. **自进化经验库**：每次任务的结果会沉淀为时间线和证据链，持续更新知识库，避免重复犯错。
4. **多客户端兼容**：支持 Claude Code、Cursor、Cline、Codex CLI 等主流 AI 编码工具。
5. **标准化报告输出**：从证据到发现再到路径，自动生成结构化安全测试报告。

适用场景：

主要面向安全研究员、渗透测试工程师、CTF 选手，以及正在用 AI 辅助做代码审计或漏洞分析的人。
典型场景包括：用 Claude Code 分析一个恶意 APK、用 Cursor 调试前端加密逻辑、或在 Kali 上让 AI 协助做授权渗透测试。

技术亮点：

核心是“规则驱动 + 状态机”的设计——通过 MASTER-ROUTING.md 和 PowerShell 脚本构建主路由，用 scope.md 管理授权与网络配置，确保 AI 在授权确认前不会对目标发起操作。另外，跨平台工

具索引刷新脚本（Windows/Linux/Kali）和 MCP 服务器集成，让 AI 能动态感知本机工具能力，而不是硬编码路径。

上手建议：

先克隆仓库，按平台运行工具索引刷新脚本（如 `powershell -File skills/scripts/refresh-tool-index.ps1`），然后让 AI 阅读 `README_AI.md` 完成自启动配置。想贡献的话，可以从补充 `skills/routing.md` 里的场景路由矩阵，或优化某个平台（如 Kali）的工具检测脚本开始。

#3

Rust

连续 2 天

pdf-inspector

Fast Rust library for PDF inspection, classification, and text extraction. Intelligently detects scanned vs text-based PDFs to enable smart routing decisions.

Stars **9,237** Forks **614** Today **+1,699**

<https://github.com/firecrawl/pdf-inspector>

项目简介：

pdf-inspector 是 Firecrawl 开源的一个 Rust 库，专注于 PDF 的快速分类与文本提取。它能在无需 OCR 的情况下，智能判断 PDF 是文本型还是扫描型，并据此帮助开发者决定是否值得调用昂贵的 OCR 服务，从而节省资源和时间。

核心功能：

- **智能分类**：在 10-50ms 内识别 PDF 类型（文本/扫描/图像/混合），并给出置信度评分，支持按页路由。
- **高质量文本提取**：支持位置感知提取、多栏阅读顺序、RTL 文本，以及 CID 字体（含 ToUnicode 映射）的复杂编码。
- **Markdown 转换**：自动识别标题、列表、代码块、表格、粗斜体、链接和分页，输出结构化文档。
- **表格检测**：通过绘图指令矩形和文本对齐启发式两种模式，处理跨页金融表格和脚注。
- **多语言绑定**：提供 Python、Node.js 和浏览器 WebAssembly 接口，灵活集成到不同技术栈。

适用场景：

适合需要快速处理大量原生文本 PDF 的团队，如文档解析管道、RAG 系统、法律/金融报告分析工具，以及任何希望在本机 200ms 内完成 PDF 结构化提取、避免 OCR 成本和延迟的应用。也是那些需要精确阅读顺序和表格结构的 Markdown 转换工具的理想底层。

技术亮点：

- 纯 Rust 实现，仅依赖 lopdf，无 ML 模型，轻量且性能卓越（200 份文档仅 0.47s）。
- 在基准测试中，阅读顺序和表格准确率显著优于 PyMuPDF4LLM 和 MarkItDown，整体得分最高。
- 支持浏览器端 Wasm 运行，数据无需离开本地，适合隐私敏感场景。
- 自动检测字体编码异常并标记，方便调用方回退到 OCR，避免静默错误。

上手建议：

- 若想快速试用，可直接通过 pip 安装 Python 包或使用 npm 安装 Node 包，调用 process_pdf 即可

获得类型和 Markdown 输出。

- 想深入贡献，可从阅读 `docs/python.md` 和 `napi/README.md` 开始，了解 API 设计；随后可参与 benchmark 对比测试（见 `docs/benchmarking.md`），或为表格检测、CID 字体支持等模块提交改进。

- 项目活跃度高（今日新增 1.7k 星标），社区和文档都在快速完善，适合作为学习 Rust 解析器设计的优秀参考案例。

#4

Python

NEW

ADR

ADR secures enterprise AI agents through observability, security benchmarking, and threat detection. Deployed at Uber.

Stars **511** Forks **60** Today **+140**

<https://github.com/uber/ADR>

好的，以下是对 Uber 开源的 ADR 项目的深度解读：

项目简介：ADR (Agentic AI Detection and Response) 是 Uber 开源的企业级 AI 智能体安全防护系统。它专门解决员工或客户在使用 AI 编程助手（如 Cursor、Claude Code）和 AI 客服等智能体时，可能产生的数据泄露、恶意指令注入等安全问题。该项目已在 Uber 生产环境部署，并已被 MLSys 2026 会议收录。

核心功能：该项目包含四大核心能力：一是可观测性，能跨平台（macOS/Linux/Windows）采集 7 种以上 AI 编程工具的执行轨迹与意图；二是安全基准测试，提供了覆盖 17 种攻击技术的 300+ 任务和 133 个 MCP 服务器；三是威胁检测，采用“高召回初筛 + 深度推理”的双层架构高效识别风险行为；四是风险预防，该组件尚未开源，但值得期待。

适用场景：主要面向两类组织：一是使用 AI 编程工具的企业研发团队，需要防止代码或内部信息通过 AI 助手泄露；二是部署了 AI 客服或内部自动化智能体的企业，需要实时监测和阻断恶意指令注入攻击。对 AI 安全研究员和 MCP 生态开发者也有重要参考价值。

技术亮点：最值得关注的是其“双层检测器”设计——先用轻量模型对全部会话做高召回率粗筛，仅对可疑会话调用更强的智能体进行深度推理，在保证安全性的同时大幅降低算力成本。此外，其基准测试内置了模拟伪造凭证和提示注入的合成环境，为防御性安全研究提供了标准化评估平台。

上手建议：建议从 Detection 模块入手，按照 README 指引通过 `uv sync` 安装依赖，先使用 `llamafirewall` 检测器跑通无密钥冒烟测试，再逐步尝试完整的 ADR 双智能体检测器。若想深入理解论文数据，可按照 `docs/REPRODUCIBILITY.md` 的步骤复现基准测试和图表。

#5

Shell

NEW

superpowers

An agentic skills framework & software development methodology that works.

Stars **266,056** Forks **23,787** Today **+617**

<https://github.com/obra/superpowers>

项目简介：Superpowers 是一套为 AI 编程代理（如 Claude Code、Cursor 等）设计的软件开发方法论框架。它通过一组可组合的技能和初始指令，让 AI 代理在编码前先进行需求梳理、方案设计和任务规划，而不是直接盲目写代码。

核心功能：让 AI 自动触发“需求澄清 → 规格确认 → 实施计划 → 子代理驱动开发”的完整流程；强调真正的红/绿 TDD 测试驱动开发、YAGNI 和 DRY 原则；支持多个主流 AI 编码工具（Claude Code、Cursor、Codex 等）的插件化安装；可让 AI 代理自主连续工作数小时而不偏离既定计划。

适用场景：适合使用 AI 编程助手进行中大型软件项目开发的团队和个人开发者。尤其适合那些希望 AI 从“代码生成器”升级为“有纪律的软件工程师”的场景，比如需要严格测试覆盖、需求可追溯、减少返工的项目。

技术亮点：其核心创新在于“子代理驱动开发”（subagent-driven development）——由主代理分解任务，子代理执行并互相审查代码，形成类似真实团队的分工协作。同时，技能（skills）以可组合的模块化方式触发，无需用户手动干预，大幅降低了 AI 编程的随机性和失控风险。

上手建议：根据你使用的编程工具（如 Claude Code、Cursor 等），查看 README 中对应的安装章节，一键安装插件即可。若要贡献代码，可从阅读 docs/ 目录下的方法论文档开始，理解其技能触发机制和流程设计，再提交改进意见或新的技能模块。

#6

Jupyter Notebook

连续 3 天

generative-ai-for-beginners

21 Lessons, Get Started Building with Generative AI

Stars **115,992** Forks **61,510** Today **+775**

<https://github.com/microsoft/generative-ai-for-beginners>

项目简介：这是微软官方推出的生成式 AI 入门教程仓库，通过 21 节系统课程帮助零基础开发者掌握构建生成式 AI 应用的核心技能。它解决了初学者面对 AI 技术门槛高、资源分散的问题，提供了一条清晰的学习路径。

核心功能：一是提供 21 节结构化的图文+代码课程，覆盖从提示工程到 RAG 模式的完整知识体系；二是所有示例均基于 Jupyter Notebook，可交互式运行，边学边练；三是支持 50+ 种语言的自动翻译，极大降低了非英语开发者的学习门槛；四是配套 Discord 社区，学习者可实时交流提问。

适用场景：适用于想转型 AI 应用开发的软件工程师、希望将 AI 能力融入产品的创业者，以及高校教学场景。无论是想快速上手 LLM API 调用，还是想系统理解生成式 AI 的原理，都能从中获益。

技术亮点：课程内容紧跟行业前沿，涵盖 RAG（检索增强生成）、Agent 智能体、Fine-tuning 微调等热门主题；代码示例基于 Azure OpenAI 和主流开源模型，实用性强；翻译工作通过 GitHub Action 自动化完成，保证多语言版本持续同步更新，这一工程化实践值得借鉴。

上手建议：直接按课程顺序从第 1 课开始学习即可，每课包含理论讲解和可运行的 Notebook 代码。建议先阅读 README 中的课程大纲，结合自身基础选择速览或精读。想贡献的话，可以从提交 issue 反馈翻译错误或补充新课程案例入手，社区非常欢迎 PR。

#7

TypeScript

NEW

cypress

Fast, easy and reliable testing for anything that runs in a browser.

Stars **50,663** Forks **3,621** Today **+6**

<https://github.com/cypress-io/cypress>

项目简介：Cypress 是一个基于 TypeScript 开发的前端端到端测试框架，旨在为“所有能在浏览器中运行的东西”提供快速、简单且可靠的测试方案。它直接解决了传统测试工具（如 Selenium）在速度、稳定性和开发者体验上的痛点，让前端测试变得像编写业务代码一样自然。

核心功能：一是**时间旅行调试**，测试运行时可以自动录制每一步的 DOM 快照，方便回溯问题；二是**自动等待机制**，内置的等待逻辑无需手动编写 sleep 或 retry 代码，从根源上消除测试中的竞态条件；三是**网络层控制**，可以轻松 stub 或 mock 网络请求，模拟各种后端响应场景；四是**实时重载**，代码修改后测试用例自动重新运行，体验接近前端热更新开发。

适用场景：最典型的场景是**现代 Web 应用（SPA）** 的端到端测试，尤其是使用 React、Vue 或 Angular 构建的项目。无论是开发者在本地提交代码前的快速自测，还是 CI/CD 流程中的回归测试，Cypress 都表现出色。对于需要频繁与用户交互的复杂页面（如后台管理系统、电商平台），它的价值尤为明显。

技术亮点：与 Selenium 等“驱动浏览器”的工具不同，Cypress 的架构是**直接在浏览器内运行**，与应用的 JS 代码共享同一个执行环境。这种“同进程”设计让它能直接监听和干预 DOM 事件、网络请求，从而实现了无 flake 的自动等待和网络层控制。同时，它通过代理机制处理跨域问题，让测试代码无需为跨域配置烦恼。

上手建议：对于使用者，建议直接阅读官方文档的“Installing”章节，通过 `npm install cypress --save-dev` 安装后，运行 `npx cypress open` 打开交互式界面，跟随内置示例体验核心流程。对于想贡献代码的开发者，可以从阅读 CONTRIBUTING.md 开始，了解仓库的模块划分（如 driver、runner、server），并关注 develop 分支的 CI 状态，从“good first issue”标签的 issue 入手。

#8

Jupyter Notebook

连续 3 天

airllm

AirLLM 70B inference with single 4GB GPU

Stars **27,941** Forks **3,026** Today **+1,085**<https://github.com/lyogavin/airllm>

项目简介：AirLLM 是一个旨在大幅降低大语言模型推理内存占用的开源项目，其核心卖点是让 70B 级别的超大模型能在仅有 4GB 显存的单张 GPU 上运行，且无需量化、蒸馏或剪枝。项目通过创新的内存管理技术，将推理门槛从“多卡旗舰”拉低至“入门显卡”，并持续支持了从 Llama 3.1 405B 到 Kimi K3 2.8T 等不断刷新的超大模型。

核心功能：首先，它支持多种主流模型架构（如 Llama、Qwen、DeepSeek、Mixtral 等），并提供统一的 AutoModel 接口，简化了调用流程。其次，项目实现了模型分块加载与流式推理，通过将模型层或专家（Expert）按需从磁盘加载至显存，打破了显存容量的物理限制。此外，AirLLM 还提供了模型压缩功能（宣称可带来 3 倍推理速度提升）以及 8bit/4bit 量化选项，并支持 CPU 推理和 MacOS 环境，极大扩展了适用硬件范围。

适用场景：该项目的首要受众是硬件资源有限的个人开发者、研究人员和学生，他们希望在消费级显卡上体验或微调超大参数模型。其次，它也适用于需要本地化、私有化部署大模型，但又不具备昂贵多卡服务器环境的中小企业。对于需要快速验证模型效果或进行教学演示的场景，AirLLM 也提供了极大的便利。

技术亮点：其最核心的技术在于“层流式”加载机制，它将模型权重按层拆分，在推理时仅将当前计算所需的层加载到显存，计算完成后立即释放，从而将显存占用降至最低。针对 MoE（混合专家）模型，它更是进一步优化为“单专家流式加载”，仅加载当前 Token 路由到的专家，这使得稀疏模型（如 DeepSeek-V3）能以极低显存运行。此外，项目还引入了预取（Prefetching）机制，将模型加载与计算过程重叠，以掩盖 I/O 延迟，提升实际吞吐量。

上手建议：对于使用者，最直接的路径是通过 `pip install airllm` 安装，然后参考 README 中的快速开始代码即可运行。建议从官方提供的 Colab 示例笔记本（如运行 Llama3 70B 的示例）入手，这是最稳妥的体验方式。对于希望贡献代码的开发者，可以从 GitHub 仓库的 Issues 和 Discussions 入手，了解当前待解决的问题；由于项目大量涉及模型加载策略和显存优化，熟悉 PyTorch 底层机制和 transformers 库的开发者更容易找到切入点。

#9

JavaScript

NEW

webpack

A bundler for javascript and friends. Packs many modules into a few bundled assets. Code Splitting allows for loading parts of the application on demand. Through "loaders", modules can be CommonJs, AMD, ES6 modules, CSS, Images, JSON, Coffeescript, LESS, ... and your custom stuff.

Stars **65,885** Forks **9,520** Today **+8**

<https://github.com/webpack/webpack>

项目简介：webpack 是一个开源的 JavaScript 模块打包器，它能把项目中零散的模块文件（如 JS、CSS、图片等）打包成浏览器可直接加载的静态资源。它解决的痛点是现代前端开发中模块化代码无法被浏览器直接识别，以及资源过多导致加载性能下降的问题。

核心功能：一是支持 ES Modules、CommonJS、AMD 等多种模块规范，并能混合使用；二是代码分割（Code Splitting），允许将应用拆分成多个按需加载的 chunk，减少首屏加载时间；三是通过 Loader 机制预处理各类文件（如将 TypeScript 转译为 JS、将图片转为 Base64）；四是高度灵活的 Plugin 插件系统，几乎所有核心功能都基于此接口实现，可扩展任意自定义需求。

适用场景：主要面向使用现代前端框架（React、Vue、Angular）的开发者，以及需要构建复杂单页应用或大型多页面应用的团队。凡是涉及模块化开发、需要处理多种静态资源、或对首屏性能有优化需求的项目，webpack 都是核心构建工具。

技术亮点：其依赖图（Dependency Graph）机制在编译期静态分析所有模块依赖，从而精准控制打包产物；Loader 管道设计让文件转换流程高度可组合；插件系统基于 Tapable 事件流架构，使得 webpack 自身功能与第三方扩展都运行在同一套机制上，这种自举设计保证了架构的一致性和可扩展性。

上手建议：新用户建议从官方 "Get Started" 指南入手，先理解 entry、output、loader 和 plugin 四个核心概念，然后尝试用 webpack 打包一个简单的 JS 项目。想贡献代码的开发者，可以先阅读仓库的 CONTRIBUTING 文档，从标记为 "good first issue" 的 GitHub issue 开始，逐步熟悉其基于 Tapable 的插件开发模式。

#10

C++

NEW

spdlog

Fast C++ logging library.

Stars **29,342** Forks **5,365** Today **+9**

<https://github.com/gabime/spdlog>

项目简介：spdlog 是一个极速的 C++ 日志库，旨在为 C++ 开发者提供简单、高效、功能完备的日志记录方案。它解决了 C++ 生态中长期缺乏一个既轻量又高性能的日志库的问题，让开发者无需在性能和易用性之间做出妥协。

核心功能：spdlog 支持头部-only 和编译两种模式，提供丰富的格式化输出（基于 fmt 库）以及异步日志记录。它内置了多种日志目标（Sink），包括控制台（带颜色）、轮转文件、每日文件、系统日志等，并支持运行时动态调整日志级别和自定义扩展。

适用场景：它适用于所有需要日志记录的 C++ 项目，从简单的命令行工具到复杂的服务器端应用。无论是需要快速部署的调试程序，还是对性能有极致要求的金融交易系统、游戏引擎或嵌入式应用，spdlog 都能胜任。

技术亮点：其核心亮点在于极致的性能，通过精心设计避免不必要的锁竞争和内存分配，在基准测试中表现优异。此外，它采用模块化架构，将 Logger 与 Sink 解耦，使得扩展新的日志输出目标变得非常简单。对 fmt 库的深度集成，也让格式化功能强大且类型安全。

上手建议：如果你是新手，建议直接采用头部-only 模式，只需将 include/spdlog 目录复制到你的工程中，并确保编译器支持 C++11 即可快速开始。若想深入了解或贡献代码，可以从克隆仓库、阅读文档和示例代码开始，并关注其 GitHub 上的 Issues 和贡献指南。

数据来源: github.com/trending

报告日期: 2026-08-04

由 DeepSeek AI 提供智能分析 | 自动生成